

ProConnect — Development Reference

ProConnect — Development Reference

1. Goal

Build a **LinkedIn-like professional networking application** to develop practical full-stack skills.

Focus:

- Working features
- Clean architecture
- REST APIs
- React development
- Authentication
- Database
- Basic production practices

Not the goal: building a massive enterprise LinkedIn clone.

2. Services

#	Service	Scope
1	User	Register, login, account, roles, status
2	Profile	Bio, skills, education, experience, photo
3	Networking	Send/accept/reject/remove connections
4	Post	Create/edit/delete, feed, likes, comments
5	Notification	Create, read/unread, email/push later
6	Messaging	Conversations, messages, read status
7	Premium	Plans, subscription, expiry
8	Jobs	Create/search jobs, apply, application status
9	Search	Users, posts, jobs, filtering
10	Admin	User management, moderation, reports
11	Music	Search, playlists, tracks, playback basics
12	GPS	Location, coordinates, current position
13	Map	Maps, routes, distance, nearby locations

3. Development rule

Every service should first be:

Basic



Working



Clean



Tested

Only add advanced functionality when there is a real reason.

No unnecessary:

Microservices

Complex event architecture

Advanced caching

AI

Elasticsearch

WebSockets

Complex DevOps

unless we specifically decide they are needed later.

4. Backend architecture

Basic structure:

Request



Route



Controller



Service



Repository / Model

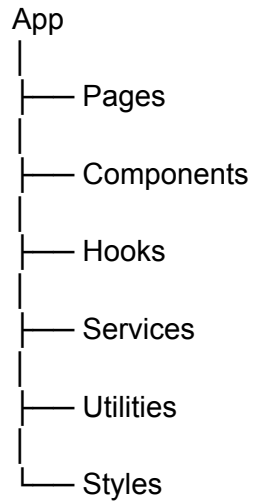


Database

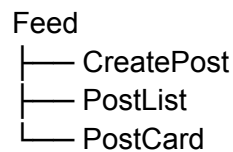
Keep responsibilities separated, but don't over-engineer.

5. Frontend architecture

React:



Example:



6. React learning goal

ProConnect is the practice environment, but React learning is independent of any single service.

We will systematically learn important React concepts:

- Components
- JSX
- Props
- State
- Events
- Forms
- Lists
- Conditional rendering
- Component composition
- State architecture
- Context
- Reducers
- Hooks

useEffect
useRef
useMemo
useCallback
Custom hooks
Routing
API communication
Authentication
Performance
Reusable architecture

For every concept:

Understand



Small example



ProConnect example



Implement

No blind coding and no unnecessary theory.

7. Backend learning goal

Node.js + Express will be developed separately, but ProConnect will give us practical implementation experience with:

- REST APIs
- Authentication
- Authorization
- CRUD
- Validation
- Database
- Relationships
- Error handling
- Middleware
- File uploads
- API security
- Testing
- Deployment basics

8. Current development position

We've already started:

- React
 - ↓
 - ProConnect Feed
 - ↓
 - Components
 - ↓
 - Props
 - ↓
 - State
 - ↓
 - Forms
 - ↓
 - useEffect
 - ↓
 - GET API
 - ↓
 - POST API
- Backend:
 - Node.js
 - ↓

Express



REST API



Posts

9. Working principle

Most important:

Don't become an expert in everything before building. Don't blindly build without understanding. Learn the required concept, understand it, implement it, and move forward.

That is the standard we'll use for **ProConnect from now on.**

Job Service

ProConnect — Job Service Development Document

Purpose: Build a simple, working Job Service that is good enough for the ProConnect project and demonstrates practical backend/frontend skills for a developer job.

Scope: Basic → Intermediate. No over-engineering.

1. What the service does

The Job Service allows users to:

1. View available jobs
2. View one job
3. Create a job
4. Edit/delete a job they created
5. Search/filter jobs
6. Apply for a job
7. View their applications
8. Track application status

2. Main data

Job

Job

- id
- title
- company
- description
- location
- employmentType
- experienceLevel
- salary
- skills
- createdBy
- createdAt

Example:

```
{
  "title": "Backend Developer",
  "company": "ProTech",
  "description": "Build REST APIs",
  "location": "Bangalore",
  "employmentType": "Full-time",
  "experienceLevel": "Mid-level",
  "salary": "8-12 LPA",
  "skills": ["Node.js", "Express", "MongoDB"]
}
```

Job Application

JobApplication

- id
- jobId
- applicantId
- resume
- coverLetter
- status
- appliedAt

3. API endpoints

Jobs

Method	Endpoint	Purpose
GET	<code>/api/jobs</code>	Get jobs
GET	<code>/api/jobs/:id</code>	Get one job
POST	<code>/api/jobs</code>	Create job
PATCH	<code>/api/jobsiuid</code>	Update own job
DELETE	<code>/api/jobs/:id</code>	Delete own job

Applications

Method	Endpoint	Purpose
POST	<code>/api/jobs/:jobId/applications</code>	Apply for job
GET	<code>/api/applications</code>	My applications
GET	<code>/api/applications/:id</code>	Application details
PATCH	<code>/api/applications/:id</code>	Update application status

4. Basic workflow

Create Job

User
↓
Create Job Form
↓
POST /api/jobs
↓
Validate data
↓
Create Job
↓
Return created job

Apply

User
↓
Job Details
↓
Apply
↓
POST /api/jobs/:jobId/applications
↓
Create application
↓
status = "Applied"

Application status

Applied
↓
Reviewing
↓
Shortlisted
↓
Hired / Rejected

Keep these statuses simple.

5. Basic validation

Required fields:

title
company
description
location
employmentType
experienceLevel

For application:

jobId
applicantId

Prevent:

- empty title
- empty description
- applying to a nonexistent job
- applying to the same job twice

6. Important authorization

A user should **not** be able to modify another user's job.

For example:

User A creates Job 101

User B

↓

PATCH /api/jobs/101

↓

 Not allowed

The backend checks:

job.createdBy === loggedInUser.id

Similarly, application status should only be changed by the appropriate job owner/admin—not by the applicant arbitrarily.

7. React implementation

Frontend pages/components:

pages/

- |— Jobs.jsx
- |— JobDetails.jsx
- |— CreateJob.jsx
- |— MyApplications.jsx

components/

- |— JobCard.jsx
- |— JobList.jsx
- |— JobForm.jsx
- |— ApplicationCard.jsx

Basic flow:

Jobs

↓

JobList

↓

JobCard

↓

JobDetails

↓

Apply

This gives us useful React practice with:

- Components
- Props
- State
- Forms
- Lists
- Conditional rendering
- API calls
- Routing

8. Search & filtering

Keep it simple.

Example:

GET /api/jobs?search=react&location=bangalore

Frontend:

Search input



React state



API request



Filtered jobs

Don't introduce a separate search engine.

9. Suggested folder structure

Backend

```
job/
├── job.routes.js
├── job.controller.js
├── job.service.js
├── job.model.js
└── job.validation.js
```

Applications can remain part of the Job Service initially:

```
job/
├── application.controller.js
├── application.service.js
└── application.model.js
```

Frontend

```
src/
├── pages/
│   ├── Jobs.jsx
│   ├── JobDetails.jsx
│   ├── CreateJob.jsx
│   └── MyApplications.jsx
├── component/
│   ├── JobCard.jsx
│   ├── JobList.jsx
│   ├── JobForm.jsx
│   └── ApplicationCard.jsx
└── service/
    └── jobService.js
```

10. Development order

Build it in this order:

1. Job model/data
- ↓
2. GET jobs
- ↓
3. GET single job
- ↓
4. POST create job
- ↓
5. PATCH job
- ↓
6. DELETE job
- ↓
7. React job list
- ↓
8. React job details
- ↓
9. Create Job form
- ↓
10. Search/filter
- ↓
11. Apply
- ↓
12. My Applications
- ↓
13. Application status

Stop when this works.

11. Interview points this service gives you

If asked **"Explain your Job Service"**, you should be able to say:

"I implemented a Job Service with CRUD operations for jobs and a separate application flow. Users can create and manage their own jobs, search and filter jobs, and apply to jobs. Applications maintain their own status, and authorization ensures users cannot modify resources they don't own."

Possible follow-up questions:

Why separate Job and JobApplication?

Because one job can have many applications:

Job

↓

Application 1

Application 2

Application 3

Why check ownership on the backend?

Because frontend restrictions are not security. The backend must verify that the authenticated user owns the resource.

Why use PATCH instead of PUT for status?

Because we're changing only part of the application resource:

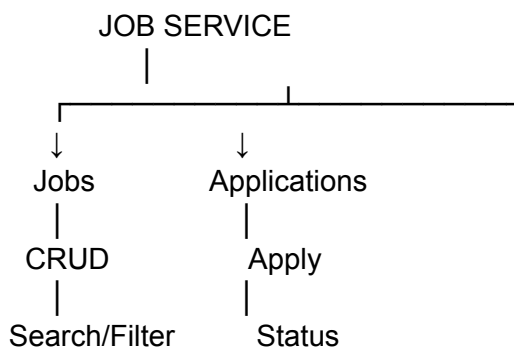
```
{  
  "status": "Shortlisted"  
}
```

How would you search for jobs?

Initially with database/API filtering using query parameters. A dedicated search engine isn't necessary for this project.

Final target

When this is finished, you should have:



That's the complete scope. Build this version first.

Search Service

ProConnect — Search Service

1. Purpose

Search across:

- Users
 - Posts
 - Jobs
-

2. Core Features

Feature	What it does
User Search	Search users by name/role/skills
Post Search	Search posts by content
Job Search	Search jobs by title/description
Basic Filtering	Filter jobs/posts where needed

3. Main API Endpoints

Method	Endpoint	Purpose
GET	/api/search/users?q=react	Search users
GET	/api/search/posts?q=react	Search posts
GET	/api/search/jobs?q=react	Search jobs
GET	/api/search?q=react	Combined search

4. Important Boundary With Other Services

User Service



owns user data

Post Service



owns post data

Job Service



owns job data

Search Service



searches these resources

Search Service **doesn't become the owner of users, posts, or jobs.**

5. Frontend Structure

pages/

└─ Search.jsx

component/

└─ SearchBar.jsx

└─ SearchResults.jsx

└─ UserResult.jsx

└─ PostResult.jsx

└─ JobResult.jsx

service/

└─ searchService.js

6. Development Order

1. Search API



2. User search



3. Post search



4. Job search



5. Combined search



6. Search React UI



7. Connect UI with API

7. Interview Perspective

You should be able to explain:

"The Search Service provides a common search API for users, posts, and jobs. Those services own their respective data, while Search is responsible for querying and returning matching results."

Scope

Working + interview-ready.

No Elasticsearch, vector search, AI search, distributed search, or other advanced additions in this document.

This is the format I'll use for the remaining ProConnect service documents.

Admin Service

ProConnect — Admin Service Development Document

1. Purpose

Admin Service allows an authorized admin to manage the platform.

Scope: basic, working, and interview-ready.

2. Core Features

Feature	What it does
User Management	View, activate, deactivate users
Moderation	Handle inappropriate posts/content
Reports	View and manage user/content reports

3. Main API Endpoints

User Management

Method	Endpoint	Purpose
GET	<code>/api/admin/users</code>	List users
GET	<code>/api/admin/users/:id</code>	View user
PATCH	<code>/api/admin/users/:id/status</code>	Activate/deactivate user

Example:

```
{
  "status": "suspended"
}
```

Moderation

Method	Endpoint	Purpose
GET	<code>/api/admin/posts</code>	View posts for moderation
DELETE	<code>/api/admin/posts/:id</code>	Remove inappropriate post

Reports

Method	Endpoint	Purpose
POST	<code>/api/reports</code>	User reports content/user
GET	<code>/api/admin/reports</code>	Admin views reports
PATCH	<code>/api/admin/reports/:id</code>	Update report status

Report status:

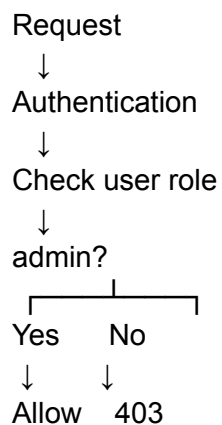
pending
reviewed
resolved

4. Important Authorization

Only admins can access:

`/api/admin/*`

Basic flow:



The frontend should hide admin pages from normal users, **but the backend must enforce the permission.**

5. Frontend Structure

```
pages/  
└─ admin/  
    ├── AdminDashboard.jsx  
    ├── Users.jsx  
    ├── Reports.jsx  
    └── Moderation.jsx
```

```
component/  
└─ UserTable.jsx  
└─ ReportCard.jsx  
└─ ModerationCard.jsx
```

```
service/  
└─ adminService.js
```

6. Development Order

1. Admin authentication/role check
↓
 2. List users
↓
 3. Activate/deactivate user
↓
 4. View reports
↓
 5. Update report status
↓
 6. Basic content moderation
↓
 7. Admin React UI
-

7. Interview Perspective

Be able to explain:

"I implemented role-based access for the Admin Service. Admins can manage user status, review reports, and moderate content. Authorization

is enforced on the backend rather than relying only on frontend route protection."

The important interview concept here is:

Authentication tells us who the user is. Authorization determines whether that user is allowed to perform the admin operation.

Premium Service

ProConnect — Premium Service Development Document

1. Purpose

Premium Service manages paid membership plans and a user's subscription status.

Scope: basic, working, and interview-ready.

2. Core Features

Feature	What it does
Plans	Show available premium plans
Subscribe	User selects a plan
Subscription	Store user's active subscription
Expiry	Check whether subscription has expired
Premium Access	Allow access to premium features

3. Main Data

Plan

Plan

- id
- name
- price
- duration
- features

Example:

```
{  
  "name": "Premium",  
  "price": 499,  
}
```

```
"duration": 30,  
"features": [  
  "Advanced profile visibility",  
  "Premium badge"  
]  
}
```

Subscription

Subscription

- id
- userId
- planId
- startDate
- expiryDate
- status

Status:

active
expired

4. Main API Endpoints

Method	Endpoint	Purpose
GET	/api/plans	Get available plans
GET	/api/subscriptions	Get current user's subscription
POST	/api/subscriptions	Subscribe to a plan
DELETE	/api/subscriptions	Cancel subscription

For the initial project, **we can simulate payment** rather than integrating a real payment gateway.

5. Subscription Flow

User



View Plans

↓
Select Plan
↓
Subscribe
↓
Create Subscription
↓
startDate + expiryDate
↓
status = active

When the expiry date passes:

Current Date > expiryDate
↓
expired
↓
Premium access removed

6. Important Boundary

Premium Service

↓
Plans + subscription status

User Service

↓
User/account information

Premium should not own general user/account data.

7. Frontend Structure

pages/

├── Premium.jsx
└── Subscription.jsx

component/

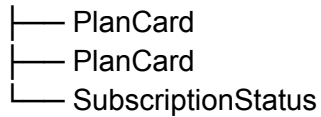
├── PlanCard.jsx
└── SubscriptionStatus.jsx

service/

└── premiumService.js

Basic UI:

Premium



8. Development Order

1. Create plans
 - ↓
 2. GET plans API
 - ↓
 3. Display plans in React
 - ↓
 4. Subscribe to plan
 - ↓
 5. Store subscription
 - ↓
 6. Check expiry
 - ↓
 7. Show premium/expired state
-

9. Interview Perspective

Be able to explain:

"I implemented a Premium Service that manages plans and user subscriptions. A subscription has a start date, expiry date, and status. Premium access is determined by the subscription status and expiry date."

Important interview concept:

Don't trust the frontend to decide whether a user is premium. The backend should verify the subscription before allowing protected premium operations.

Messaging Service

ProConnect — Messaging Service Development Document

Goal: Build a basic, working messaging system that is good for the project and demonstrates practical development skills.

No WebSocket/real-time implementation for this version. We'll use **REST APIs**.

1. Purpose

Users can:

- See their conversations
 - Open a conversation
 - Send messages
 - View messages
 - Track read/unread status
-

2. Core Features

Feature	What it does
Conversations	List conversations for the user
Messages	Send and retrieve messages
Read Status	Track whether a message has been read
Unread Count	Show unread messages

3. Main Data

Conversation

Conversation

- id
- participants

└─ updatedAt

Example:

```
{
  "id": 101,
  "participants": [12, 25],
  "updatedAt": "2026-08-22T10:30:00Z"
}
```

Message

Message

├─ id
├─ conversationId
├─ senderId
├─ content
├─ isRead
└─ createdAt

Example:

```
{
  "id": 501,
  "conversationId": 101,
  "senderId": 12,
  "content": "Hello, how are you?",
  "isRead": false,
  "createdAt": "2026-08-22T10:35:00Z"
}
```

4. Main API Endpoints

Conversations

Method	Endpoint	Purpose
GET	/api/conversations	Get user's conversations
GET	/api/conversations/:id	Get conversation details
POST	/api/conversations	Create/start conversation

Messages

Method	Endpoint	Purpose
GET	<code>/api/conversations/:id/messages</code>	Get messages
POST	<code>/api/conversations/:id/messages</code>	Send message
PATCH	<code>/api/messages/:id/read</code>	Mark message as read

5. Message Flow

User A



POST `/api/conversations/101/messages`



Messaging Service



Save Message



Return message

User B later requests:

GET `/api/conversations/101/messages`

and receives the messages.

6. Read Status

When a message is initially created:

`isRead = false`

When the receiver reads it:

PATCH `/api/messages/501/read`

Then:

`isRead = true`

Unread count can be calculated from unread messages.

7. Important Boundary

User Service



owns users

Messaging Service



owns conversations + messages

Messaging only needs the user's ID to identify participants.

8. Frontend Structure

pages/

└─ Messages.jsx

component/

├─ ConversationList.jsx
├─ ConversationItem.jsx
├─ ChatWindow.jsx
├─ MessageList.jsx
├─ MessageBubble.jsx
└─ MessageInput.jsx

service/

└─ messageService.js

UI:

Messages

├─ ConversationList
│ └─ ConversationItem
│ └─ ConversationItem
└─ ChatWindow
 └─ MessageList
 └─ MessageBubble
 └─ MessageBubble
└─ MessageInput

9. Development Order

1. Conversation data
↓
 2. Create conversation
↓
 3. Get conversations
↓
 4. Send message
↓
 5. Get messages
↓
 6. Read status
↓
 7. Build React messaging UI
↓
 8. Connect UI to REST APIs
-

10. Interview Perspective

Be able to explain:

"I implemented messaging using REST APIs. Conversations contain participants, and messages belong to a conversation. Messages initially have an unread state and can be marked as read by the receiver."

A useful interview point:

"For the initial version I used REST APIs rather than WebSockets. This keeps the implementation simple and is sufficient for basic message sending and retrieval. Real-time communication would be a separate concern."

That keeps the Messaging Service **workable, understandable, and aligned with your current job-oriented level.**

Notification Service

ProConnect — Notification Service Development Document

Goal: Basic, working notification system suitable for the project and job interviews.

1. Purpose

Notify users when something important happens in ProConnect.

Examples:

- Someone sends a connection request
 - Someone likes your post
 - Someone comments on your post
 - Someone accepts your connection request
-

2. Core Features

Feature	What it does
Create Notification	Create a notification for a user
List Notifications	Show user's notifications
Read/Unread	Track whether notification was read
Mark as Read	Mark notification as read

For this version, notifications are stored and retrieved through the REST API.

3. Notification Data

Notification

```
|— id
|— userId
|— type
|— message
|— isRead
|— createdAt
```

Example:

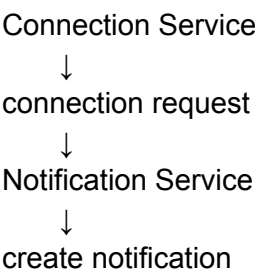
```
{
  "id": 101,
  "userId": 25,
  "type": "connection_request",
  "message": "Rahul sent you a connection request",
  "isRead": false,
  "createdAt": "2026-08-22T10:30:00Z"
}
```

4. Main API Endpoints

Method	Endpoint	Purpose
GET	<code>/api/notifications</code>	Get user's notifications
GET	<code>/api/notifications/unread</code>	Get unread notifications
PATCH	<code>/api/notifications/:id/read</code>	Mark notification as read
PATCH	<code>/api/notifications/read-all</code>	Mark all as read

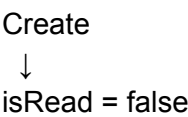
Internal creation can be handled by the service when another feature triggers a notification.

Example:



5. Read/Unread Flow

New notification:



User opens it:

PATCH /api/notifications/101/read



isRead = true

Frontend can display:

Unread → highlighted

Read → normal

6. Important Boundary

Post Service



owns posts

Connection Service



owns connections

Notification Service



owns notifications

Other services **trigger** notifications, but Notification Service manages the notification data.

7. Frontend Structure

pages/

└─ Notifications.jsx

component/

├─ NotificationList.jsx

└─ NotificationItem.jsx

service/

└─ notificationService.js

UI:

Notifications



NotificationList



NotificationItem

8. Development Order

1. Notification model
 - ↓
 2. Create notification
 - ↓
 3. Get notifications
 - ↓
 4. Show unread state
 - ↓
 5. Mark as read
 - ↓
 6. Mark all as read
 - ↓
 7. React notification UI
 - ↓
 8. Connect with other services
-

9. Interview Perspective

Be able to explain:

"I created a Notification Service that stores user notifications and maintains their read/unread state. Other services such as Connection or Post can trigger notification creation, while the Notification Service owns notification data."

Important distinction:

Creating a notification and delivering a notification are different responsibilities.

For our current version, we're only implementing **stored/in-app notifications**. Email and push notifications are **not part of this implementation**.

Post Service

ProConnect — Post Service Development Document

Goal: Build a basic, working posting system with enough functionality to demonstrate real backend + React development.

1. Purpose

Users can:

- Create posts
 - View feed
 - View a single post
 - Edit their own post
 - Delete their own post
 - Like/unlike posts
 - Add comments
 - View comments
-

2. Core Features

Feature	What it does
Create Post	User publishes a post
Feed	Display posts
Post Details	View one post
Edit Post	Author edits own post

Delete Post **Author deletes own post**

Like **Like/unlike a post**

Comment **Add comment**

Comments **View comments**

3. Main Data

Post

Post

|— id

|— authorId

|— content

|— likesCount

|— createdAt

Example:

```
{  
  "id": 101,  
  "authorId": 25,  
  "content": "Learning React with ProConnect",  
  "likesCount": 10,  
  "createdAt": "2026-08-22T10:30:00Z"  
}
```

Comment

Comment

- |— id
 - |— postId
 - |— authorId
 - |— content
 - |— createdAt
-

4. Main API Endpoints

Posts

Method	Endpoint	Purpose
GET	/api/posts	Get feed
GET	/api/posts/:id	Get single post
POST	/api/posts	Create post
PATCH	/api/posts/:id	Edit own post
DELETE	/api/posts/:id	Delete own post

Likes

Method	Endpoint	Purpose
POST	<code>/api/posts/:id/like</code>	Like post
DELETE	<code>/api/posts/:id/like</code>	Unlike post
Comments		
Method	Endpoint	Purpose
GET	<code>/api/posts/:id/comments</code>	Get comments
POST	<code>/api/posts/:id/comments</code>	Add comment
DELETE	<code>/api/comments/:id</code>	Delete own comment

5. Create Post Flow

CreatePost



POST `/api/posts`



Post Service



Validate content



Create Post



Return post



Update React state

6. Feed Flow

Feed



GET /api/posts



PostList



PostCard

React structure:

Feed

|— CreatePost

|— PostList

 |— PostCard

7. Edit/Delete Ownership

A user can only modify their own post.

User A



Post created by User A



Edit/Delete



Allowed

User B



Post created by User A



Edit/Delete



403 Forbidden

Backend must perform this check.

8. Like Flow

User clicks Like



POST /api/posts/:id/like



Create user's like



likesCount changes



Return updated result

The same user should not be able to create multiple likes for the same post.

9. Comment Flow

User writes comment



POST /api/posts/:id/comments



Create Comment



Return comment



React updates comment list

10. Important Boundary

Post Service

├— Posts

├— Likes

└— Comments

User information remains owned by:

User Service

The Post Service only needs **authorId**.

11. Frontend Structure

pages/

└— Feed.jsx

component/

|— **CreatePost.jsx**

|— **PostList.jsx**

|— **PostCard.jsx**

|— **CommentList.jsx**

|— **CommentItem.jsx**

|— **CommentForm.jsx**

service/

|— **postService.js**

12. Development Order

1. Create Post

↓

2. Get Feed

↓

3. Post Details

↓

4. Edit Post

↓

5. Delete Post

↓

6. Like / Unlike

↓

7. Comments

↓

8. React UI integration

13. Interview Perspective

Be able to explain:

"I implemented a Post Service supporting CRUD operations, likes, and comments. Posts belong to users through **authorId**, and authorization checks ensure users can only modify their own content."

Important interview concepts from this service:

- REST CRUD
- Resource ownership
- Authorization
- Relationships
- State updates in React
- Preventing duplicate likes
- Parent-child data flow for comments

This is the complete scope for the first working Post Service.

Networking Service

ProConnect — Networking Service Development Document

Goal: Build a basic, working connection system like LinkedIn's core networking feature.

1. Purpose

Users can:

- Send connection requests
 - Accept requests
 - Reject requests
 - Remove existing connections
 - View their connections
 - See connection status
-

2. Core Features

Feature	What it does
Send Request	Send connection request to another user
Accept Request	Accept incoming request
Reject Request	Reject incoming request
Remove Connection	Remove an existing connection
My Connections	List connected users
Connection Status	Check current relationship

3. Main Data

Connection

Connection

- id
- senderId
- receiverId
- status
- createdAt

Status:

pending
accepted
rejected

For a removed connection, delete the connection record rather than adding another status.

4. Main API Endpoints

Method	Endpoint	Purpose
POST	<code>/api/connections/:userId/request</code>	Send request
PATCH	<code>/api/connections/:id/accept</code>	Accept request
PATCH	<code>/api/connections/:id/reject</code>	Reject request
DELETE	<code>/api/connections/:id</code>	Remove connection
GET	<code>/api/connections</code>	Get my connections
GET	<code>/api/connections/:userId/status</code>	Get connection status

5. Connection Flow

Send request

User A

↓
Send Connection
↓
POST /api/connections/UserB/request
↓
Create connection
↓
status = pending

Accept

User B
↓
Accept
↓
PATCH /api/connections/:id/accept
↓
status = accepted

Reject

User B
↓
Reject
↓
PATCH /api/connections/:id/reject
↓
status = rejected

6. Important Rules

Don't connect to yourself

User A → User A



Don't send duplicate requests

A → B
pending

A → B again



Only receiver can accept/reject

A sends request to B

A → accept



B → accept



Only connected users can remove the connection.

These checks belong in the **backend**, not only React.

7. Important Boundary

User Service



owns user/account data

Networking Service



owns connection relationships

Networking uses **senderId** and **receiverId**; it doesn't own the user's profile information.

8. Frontend Structure

pages/

- Connections.jsx
- People.jsx

component/

- UserCard.jsx
- ConnectionButton.jsx
- ConnectionRequest.jsx
- ConnectionList.jsx

service/

- connectionService.js

Example:

People



UserCard



ConnectionButton

Button can display:

Connect
Pending
Accept
Connected
Remove

depending on the connection status.

9. Development Order

1. Send request
↓
 2. Accept request
↓
 3. Reject request
↓
 4. Remove connection
↓
 5. Get connections
↓
 6. Get connection status
↓
 7. React UI
↓
 8. Connect UI with API
-

10. Interview Perspective

Be able to explain:

"I implemented a Networking Service where a connection represents a relationship between a sender and receiver. Requests start as pending and can be accepted or rejected. Authorization ensures that only the receiver can accept or reject a request."

Important interview concepts:

- Resource relationships
- Authorization
- Preventing duplicate requests
- State transitions
- REST API design
- Database relationships

Profile Service

ProConnect — Profile Service Development Document

Goal: Build a basic, working professional profile system for ProConnect.

1. Purpose

Users can manage their professional profile:

- Bio
 - Skills
 - Education
 - Experience
 - Profile photo
-

2. Core Features

Feature	What it does
View Profile	View a user's profile
Create Profile	Create profile information
Update Profile	Edit profile information
Skills	Add/update skills
Education	Add/update education
Experience	Add/update work experience
Profile Photo	Upload/update profile photo

3. Main Data

Profile

Profile

- id
- userId

- bio
- skills
- education
- experience
- profilePhoto

Example:

```
{
  "userId": 101,
  "bio": "Backend Developer interested in building scalable APIs.",
  "skills": [
    "Python",
    "Django",
    "Node.js",
    "React"
  ],
  "education": [
    {
      "degree": "B.Tech",
      "institution": "ABC University"
    }
  ],
  "experience": [
    {
      "company": "ABC Technologies",
      "role": "Software Developer",
      "years": 2
    }
  ],
  "profilePhoto": "/uploads/profile-101.jpg"
}
```

4. Main API Endpoints

Method	Endpoint	Purpose
GET	<code>/api/profile/me</code>	Get logged-in user's profile
GET	<code>/api/profile/:userId</code>	Get another user's profile
POST	<code>/api/profile</code>	Create profile
PATCH	<code>/api/profile</code>	Update profile
PATCH	<code>/api/profile/skills</code>	Update skills
PATCH	<code>/api/profile/education</code>	Update education
PATCH	<code>/api/profile/experience</code>	Update experience
POST	<code>/api/profile/photo</code>	Upload profile photo

5. Profile Flow

View profile

Profile Page
↓
GET `/api/profile/me`
↓
Profile Service
↓
Profile data
↓
React state
↓
Display profile

Update profile

Edit Profile
↓
User changes data
↓
PATCH `/api/profile`

↓
Validate data
↓
Update profile
↓
Return updated profile

6. Important Boundary

User Service

↓
Account
Login
Roles
Status

Profile Service

↓
Bio
Skills
Education
Experience
Photo

The **Profile** uses **userId** to associate the profile with a user.

Profile Service should **not manage login/password/account authentication**.

7. Frontend Structure

pages/

├── Profile.jsx
└── EditProfile.jsx

component/

├── ProfileHeader.jsx
├── ProfileAbout.jsx
├── SkillsList.jsx
├── EducationList.jsx
├── ExperienceList.jsx
└── ProfileForm.jsx

service/

└── profileService.js

Basic structure:

Profile



8. Development Order

1. Create profile
 - ↓
 2. Get profile
 - ↓
 3. Update basic profile
 - ↓
 4. Add skills
 - ↓
 5. Add education
 - ↓
 6. Add experience
 - ↓
 7. Profile photo
 - ↓
 8. React Profile page
 - ↓
 9. Edit Profile page
 - ↓
 10. Connect UI with API
-

9. Interview Perspective

Be able to explain:

"I separated account information from professional profile information. The User Service manages authentication and account data, while the Profile Service manages professional information such as skills, education, experience, and profile photo."

Important concepts from this service:

- Resource ownership using `userId`
- CRUD/API design

- Form handling
- File upload
- Data relationships
- React form state
- Backend authorization

This is the complete first version of the Profile Service.

User Service

ProConnect — User Service Development Document

Goal: Build a basic, working User Service that handles user accounts and authentication.

1. Purpose

Users can:

- Register
 - Login
 - View their account
 - Update basic account information
 - Have a role
 - Have an account status
-

2. Core Features

Feature	What it does
Register	Create a new user account
Login	Authenticate user
Account	View/update basic account information
Roles	Identify user permissions
Status	Activate/deactivate account

3. Main Data

User

User

- id
- name
- email
- password

└─ role
└─ status
└─ createdAt

Example:

```
{  
  "id": 101,  
  "name": "Rahul",  
  "email": "rahul@example.com",  
  "role": "user",  
  "status": "active",  
  "createdAt": "2026-08-22T10:30:00Z"  
}
```

Passwords should never be returned in API responses.

4. Main API Endpoints

Authentication

Method	Endpoint	Purpose
POST	<code>/api/auth/register</code>	Register user
POST	<code>/api/auth/login</code>	Login user
POST	<code>/api/auth/logout</code>	Logout user

Account

Method	Endpoint	Purpose
GET	<code>/api/users/me</code>	Get logged-in user
PATCH	<code>/api/users/me</code>	Update account information

5. Register Flow

Register Form



POST /api/auth/register



Validate data



Check email



Hash password



Create User



Return user

Example request:

```
{  
  "name": "Rahul",  
  "email": "rahul@example.com",  
  "password": "secret123"  
}
```

6. Login Flow

Login Form



POST /api/auth/login



Find user



Compare password



Create authentication token



Return token

The frontend then uses the authentication token for protected requests.

7. Roles

For the first version:

user
admin

Example:

user
↓
normal ProConnect features

admin
↓
Admin Service

Role checking belongs to **authorization**, not authentication.

8. Account Status

Use:

active
suspended

Example:

active
↓
can use application

suspended
↓
blocked from protected operations

9. Important Boundary

User Service

- Account
- Authentication
- Role
- Account Status

Profile Service

- Bio
- Skills
- Education
- Experience
- Photo

For example:

User

- name
- email
- role

Profile

- bio
- skills
- experience

Don't put all professional profile information inside the User model.

10. Frontend Structure

pages/

- Login.jsx

```
|— Register.jsx
|— Account.jsx
```

component/

```
|— LoginForm.jsx
|— RegisterForm.jsx
|— AccountForm.jsx
```

service/

```
|— authService.js
```

Basic flow:

Register



Login



Authenticated User



Account

11. Development Order

1. User model



2. Register API



3. Password hashing



4. Login API



5. Authentication



6. Get current user



7. Update account



8. Role



9. Account status



10. React Login/Register



11. Connect frontend with API

12. Interview Perspective

Be able to explain:

"I implemented a User Service responsible for registration, authentication, account information, roles, and account status. Passwords are hashed before storage, and protected endpoints identify the authenticated user before performing operations."

Important concepts from this service:

- Authentication vs authorization
- Password hashing
- JWT/token-based authentication
- Protected routes
- Role-based authorization
- Account status
- Separating User from Profile data

This is the complete first version of the User Service.

Internship-reference-1

Goal: Build a simple working music feature inside ProConnect.

Level: Basic/intermediate and job-oriented.

1. Purpose

Users can:

- Search music
- View a track
- Create playlists
- View playlists
- Add/remove tracks from playlists

We are **not building a music streaming platform**. The service is mainly for learning API design, React state, lists, relationships, and basic media-related UI.

2. Core Features

Feature	What it does
Music Search	Search available tracks
Track Details	View track information
Create Playlist	Create a playlist
View Playlist	View tracks in a playlist
Update Playlist	Rename playlist
Delete Playlist	Delete playlist
Add Track	Add a track to playlist
Remove Track	Remove track from playlist

3. Main Data

Track

Track

- id
- title
- artist
- album
- duration

Example:

```
{
  "id": 101,
  "title": "Song Name",
  "artist": "Artist Name",
  "album": "Album Name",
  "duration": 210
}
```

Playlist

Playlist

- id
- userId
- name
- tracks
- createdAt

Example:

```
{
  "id": 10,
  "userId": 25,
  "name": "My Favorites",
  "tracks": [101, 105, 110]
}
```

4. Main API Endpoints

Music

Method	Endpoint	Purpose
GET	<code>/api/music/search?q=</code>	Search tracks
GET	<code>/api/music/tracks/:id</code>	Get track

Playlists

Method	Endpoint	Purpose
GET	<code>/api/playlists</code>	Get user's playlists
POST	<code>/api/playlists</code>	Create playlist
GET	<code>/api/playlists/:id</code>	Get playlist
PATCH	<code>/api/playlists/:id</code>	Rename/update playlist
DELETE	<code>/api/playlists/:id</code>	Delete playlist

Playlist Tracks

Method	Endpoint	Purpose
POST	<code>/api/playlists/:id/tracks</code>	Add track
DELETE	<code>/api/playlists/:id/tracks/:trackId</code>	Remove track

Total: 9 endpoints

5. Basic Flow

Search

Music Page
↓
Search input
↓
GET `/api/music/search?q=rock`
↓
Track results
↓
TrackCard

Playlist

User
↓

Create Playlist



POST /api/playlists



Playlist created



Add Track



POST /api/playlists/:id/tracks

6. Important Boundary

Music Service

└─ Tracks

└─ Playlists

User Service

└─ User/account

The playlist belongs to a user through:

userId

The Music Service doesn't manage the user's account.

7. Frontend Structure

pages/

└─ Music.jsx

component/

└─ SearchMusic.jsx

└─ TrackList.jsx

└─ TrackCard.jsx

└─ PlaylistList.jsx

└─ PlaylistCard.jsx

└─ PlaylistDetails.jsx

service/

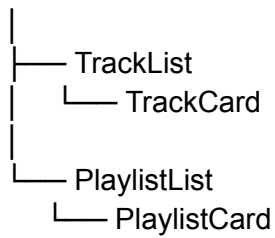
└─ musicService.js

Basic UI:

Music



└─ SearchMusic



8. Development Order

1. Track data
↓
 2. Search tracks
↓
 3. Track details
↓
 4. Create playlist
↓
 5. Get playlists
↓
 6. Add track
↓
 7. Remove track
↓
 8. Update playlist
↓
 9. Delete playlist
↓
 10. React UI
↓
 11. Connect UI with API
-

9. Interview Perspective

You should be able to explain:

"I implemented a Music Service where users can search tracks and manage playlists. A playlist belongs to a user and contains tracks. I exposed REST APIs for playlist CRUD operations and adding or removing tracks."

The useful interview concepts here are:

- REST API design
- Resource relationships
- CRUD

- Query parameters
- React lists and state
- User ownership/authorization

That's the complete scope for the first working Music Service.

Internship-reference-2

ProConnect — GPS Service Development Document

Goal: Build a simple GPS/location feature to understand how an application receives, stores, and displays a user's location.

Scope: Basic, working, and job-oriented.

1. Purpose

Users can:

- Save their current location
- Update their location
- View their saved location
- Remove their saved location

The GPS Service deals with location coordinates, not maps or route calculation.

2. Core Features

Feature	What it does
Get Location	Get user's saved coordinates
Save Location	Store latitude/longitude
Update Location	Update coordinates
Remove Location	Delete saved location

3. Main Data

Location

Location

- |— id
- |— userId
- |— latitude
- |— longitude
- |— updatedAt

Example:

```
{  
  "id": 1,  
  "userId": 101,  
  "latitude": 12.9716,  
  "longitude": 77.5946,  
  "updatedAt": "2026-08-22T10:30:00Z"  
}
```

4. Main API Endpoints

Method	Endpoint	Purpose
GET	/api/location	Get user's location

POST **/api/location** **Save/update location**

DELETE **/api/location** **Remove location**

Total: 3 endpoints

5. Basic Flow

Browser



Get user's location



latitude + longitude



POST /api/location



GPS Service



Store location

To retrieve it:

React



GET /api/location



latitude + longitude



Display location

6. Frontend Structure

pages/

└─ Location.jsx

component/

└─ LocationButton.jsx

└─ LocationDisplay.jsx

service/

└─ locationService.js

Basic flow:

Location



LocationButton



Browser Geolocation API



latitude / longitude



API

7. Important Boundary

GPS Service



Location coordinates

Map Service



Maps

Routes

Distance

Nearby locations

GPS answers:

"Where are the coordinates?"

Map answers:

"What is around these coordinates / how do I get somewhere?"

8. Development Order

1. Create location data



2. Save location API



3. Get location API



4. Delete location API



5. React Geolocation API



6. Connect frontend with backend



7. Display coordinates

9. Interview Perspective

You should be able to explain:

"I implemented a GPS Service that stores a user's latitude and longitude. The browser obtains the location using the Geolocation API, and the frontend sends the coordinates to the backend through a REST API."

Important concepts:

- **Browser Geolocation API**
- **Latitude/longitude**
- **REST API**
- **User-specific data**
- **Permissions**
- **Basic location handling**

Internship-reference-3

ProConnect — Map Service Development Document

Goal: Build a simple, working Map Service that uses coordinates to provide basic map-related operations.

Scope: Basic, practical, and interview-ready.

1. Purpose

The Map Service handles:

- Nearby locations
- Distance between two coordinates
- Basic route information

The **GPS Service** provides coordinates.
The **Map Service** uses those coordinates.

2. Core Features

Feature	What it does
Distance	Calculate distance between two locations
Nearby	Find locations near given coordinates
Route	Get basic route information

3. Main API Endpoints

Method	Endpoint	Purpose
GET	<code>/api/map/distance</code>	Calculate distance
GET	<code>/api/map/nearby</code>	Find nearby locations

GET `/api/map/route` Get route

Distance

Example:

GET /api/map/distance?lat1=12.97&lng1=77.59&lat2=13.08&lng2=80.27

Response:

```
{  
  "distance": 290  
}
```

Nearby

GET /api/map/nearby?lat=12.97&lng=77.59

Returns nearby locations.

Route

GET /api/map/route?from=12.97,77.59&to=13.08,80.27

Returns basic route information.

4. Basic Flow

Distance

Frontend
↓
coordinates
↓
GET /api/map/distance
↓
Map Service
↓
calculate distance
↓
return result

Nearby

User Location

↓
latitude + longitude
↓
Map Service
↓
find nearby locations
↓
return locations

5. Important Boundary

GPS Service
↓
Gets/stores user's coordinates

Map Service
↓
Uses coordinates
├── distance
├── nearby
└── route

So:

GPS = Where am I?

Map = What can I do with that location?

6. Frontend Structure

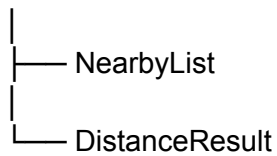
pages/
└── Map.jsx

component/
├── MapView.jsx
├── NearbyList.jsx
└── DistanceResult.jsx

service/
└── mapService.js

Basic UI:

Map
└── MapView



7. Development Order

1. Distance API
 - ↓
 2. Nearby API
 - ↓
 3. Route API
 - ↓
 4. React Map page
 - ↓
 5. Connect APIs
 - ↓
 6. Display results
-

8. Interview Perspective

You should be able to explain:

"I separated GPS and Map responsibilities. GPS handles obtaining and storing coordinates, while the Map Service uses those coordinates for operations such as calculating distance, finding nearby locations, and getting route information."

Important concepts:

- Latitude/longitude
- Query parameters
- Distance calculation
- REST API
- Separation of responsibilities
- Frontend API integration